

Windows Server Automation

1.1 Introduction

As organizations grow, manually configuring and maintaining servers becomes increasingly time-consuming, repetitive, and prone to human error. Tasks such as installing software packages, configuring secure remote access, synchronizing server data, and migrating applications often require administrators to execute numerous commands manually on each server. This approach is inefficient and can lead to inconsistent server configurations.

To address these challenges, I reproduce the server-automation approach previously developed on the Linux VMs using Windows Server, PowerShell, WinRM, and Robocopy. The implementation uses two Windows Server virtual machines:

- **Windows01** — primary/source server
- **Windows02** — secondary/destination server

The overall objective was to automate the configuration and synchronization of Windows01 with Windows02 so that important server data and required server functionality could be reproduced on the secondary server. The Windows implementation follows the same general principles as the Linux implementation:

Linux	Windows
Bash Scripts	PowerShell Scripts
SSH	WinRM / PowerShell Remoting
rsync	Rodocopy
Ubuntu packages	Windows Server roles/features
Linux file system	Windows file system

2.1 Windows Virtual Machine Environment

The Windows automation environment was implemented using two Windows Server virtual machines. The two servers were configured to communicate with each other over the same network. The purpose of separating the environment into two servers was to reproduce the primary/secondary server arrangement used in the Linux implementation where Windows01 was treated as the source server, while Windows02 was treated as the destination server. The servers were assigned the following roles:

Server	Role	IPv4 Address
Windows01	Primary/source server	192.168.0.80
Windows02	Secondary/destination server	192.168.0.81

2.2 Static IP Configuration

A reliable server-to-server automation environment requires predictable network addresses. During the initial configuration, both Windows VMs were therefore configured with static IPv4 addresses.

Windows01

IPv4 Address: 192.168.0.80

Subnet Mask: 255.255.255.0

Default Gateway: 192.168.0.1

Windows02

IPv4 Address: 192.168.0.81

Subnet Mask: 255.255.255.0

Default Gateway: 192.168.0.1

The static addresses were important because later automation relied on Windows01 being able to consistently identify Windows02 as the destination server.

2.3 Network Connectivity Testing

After configuring the static addresses, connectivity between the two Windows servers was tested before continuing with the automation. From Windows02, Windows01 was tested using:

```
ping 192.168.0.80
```

The ping test was successful. The firewall was subsequently left enabled and connectivity was tested again. The successful result confirmed that the servers could communicate without requiring the Windows firewall to be disabled.

2.4 IIS Server Configuration

The Windows environment was configured as an IIS web-server environment. IIS, or Internet Information Services, is Microsoft's web-server platform. In the Linux implementation, the web-server role was provided by Linux-based server software. In the Windows implementation, IIS provided the equivalent web-serving function. The installed Windows Server features were inspected using PowerShell:

```
Get-WindowsFeature |  
Where-Object {$_.Installed -eq $true} |  
Select-Object Name, DisplayName
```

This confirmed that IIS and its required components were installed.

2.5 Windows Services Verification

Windows services were inspected to determine whether the services required by the server environment were running. The following PowerShell command was used:

```
Get-Service |  
Where-Object {$_.Status -eq "Running"} |  
Select-Object Name, DisplayName
```

This provided a list of currently running services.

3.1 PowerShell Automation

After establishing the Windows server environment and confirming network connectivity, the next stage was to implement the Windows equivalent of the Linux Bash-based server automation.

Windows PowerShell was used as the primary automation tool. PowerShell provides commands that can directly manage Windows Server features, services, networking, files, IIS, and remote computers.

The Windows automation was organized into separate PowerShell scripts so that each script had a specific responsibility rather than placing all configuration tasks into one large script. The main scripts developed during this stage were:

- Setup-Windows.ps1
- configure_winrm.ps1
- sync_server.ps1
- install_packages.ps1

3.2 Setup-Windows.ps1

The purpose of `Setup-Windows.ps1` was to automate the initial setup of the Windows Server environment. The scripts perform four main tasks:

- Checks that PowerShell is running with administrator privileges.
- Installs and configures the OpenSSH Server.
- Installs and configures IIS.
- Verifies that both SSH and IIS services are running.

This made the initial Windows configuration repeatable instead of requiring each service to be configured manually.

3.3 Complete Script

```
# =====
# Windows Server Initial Setup
# =====

Write-Host "===== " -ForegroundColor Cyan
Write-Host " Windows Server Initial Setup" -ForegroundColor Cyan
Write-Host "===== " -ForegroundColor Cyan

Write-Host ""
Write-Host "Checking administrator privileges..." -ForegroundColor Yellow

$currentUser = [Security.Principal.WindowsIdentity]::GetCurrent()
$principal = New-Object Security.Principal.WindowsPrincipal($currentUser)

if (-not $principal.IsInRole(
    [Security.Principal.WindowsBuiltInRole]::Administrator
)) {
    Write-Host "ERROR: Please run PowerShell as Administrator." -ForegroundColor Red
    exit 1
}

Write-Host "Administrator privileges confirmed." -ForegroundColor Green
```

```

# =====
# Install OpenSSH Server
# =====

Write-Host ""
Write-Host "Checking OpenSSH Server..." -ForegroundColor Yellow

$sshCapability = Get-WindowsCapability -Online |
    Where-Object Name -like 'OpenSSH.Server*'

if ($sshCapability.State -eq "Installed") {

    Write-Host "OpenSSH Server is already installed." -ForegroundColor Green

}
else {

    Write-Host "Installing OpenSSH Server..." -ForegroundColor Yellow

    Add-WindowsCapability -Online -Name $sshCapability.Name

    Write-Host "OpenSSH Server installation completed." -ForegroundColor Green
}

# =====
# Configure OpenSSH Server
# =====

Write-Host ""
Write-Host "Configuring SSH service..." -ForegroundColor Yellow

Set-Service -Name sshd -StartupType Automatic

Start-Service -Name sshd

Write-Host "SSH service started successfully." -ForegroundColor Green

# =====
# Install IIS
# =====

Write-Host ""
Write-Host "Checking IIS..." -ForegroundColor Yellow

$iisFeature = Get-WindowsFeature -Name Web-Server

if ($iisFeature.Installed) {

    Write-Host "IIS is already installed." -ForegroundColor Green

}
else {

    Write-Host "Installing IIS..." -ForegroundColor Yellow

    Install-WindowsFeature -Name Web-Server -IncludeManagementTools

    Write-Host "IIS installation completed." -ForegroundColor Green
}

```

```

# =====
# Configure IIS
# =====

Write-Host ""
Write-Host "Configuring IIS service..." -ForegroundColor Yellow

Set-Service -Name W3SVC -StartupType Automatic

Start-Service -Name W3SVC

Write-Host "IIS service started successfully." -ForegroundColor Green


# =====
# Verify Services
# =====

Write-Host ""
Write-Host "Checking installed services..." -ForegroundColor Yellow

$sshStatus = Get-Service -Name sshd
$iisStatus = Get-Service -Name W3SVC

Write-Host ""
Write-Host "OpenSSH Server: $($sshStatus.Status)" -ForegroundColor Cyan
Write-Host "IIS:                $($iisStatus.Status)" -ForegroundColor Cyan


# =====
# Final Result
# =====

if (
    $sshStatus.Status -eq "Running" -and
    $iisStatus.Status -eq "Running"
) {

    Write-Host ""
    Write-Host "===== " -ForegroundColor Green
    Write-Host " Setup completed successfully." -ForegroundColor Green
    Write-Host "===== " -ForegroundColor Green

}
else {

    Write-Host ""
    Write-Host "===== " -ForegroundColor Red
    Write-Host " Setup completed with service errors." -ForegroundColor Red
    Write-Host "===== " -ForegroundColor Red

}

```

3.4 How to run Setup-Windows.ps1

The script should be executed from an Administrator PowerShell window. First, navigate to the automation directory:

```
cd C:\ServerAutomation
```

Confirm that the script exists:

```
Get-ChildItem
```

Then execute it:

```
.\Setup-Windows.ps1
```

The script displays its progress in the PowerShell console.

3.5 configure_winrm.ps1

The second script was designed to configure PowerShell Remoting between the Windows servers. Its purpose is to prepare WinRM, configure the required firewall rule, configure the trusted remote server, and test the connection. This provides the Windows equivalent of the remote-management functionality provided by SSH in the Linux environment.

3.6 Complete Script

```
# Configure PowerShell Remoting / WinRM

Write-Host "======"
Write-Host " Windows PowerShell Remoting Configuration"
Write-Host "======"

param(
    [Parameter(Mandatory=$true)]
    [string]$PeerIP
)

Write-Host ""
Write-Host "Enabling PowerShell Remoting..."

Enable-PSRemoting -Force

Write-Host ""
Write-Host "Configuring Windows Firewall..."

Enable-NetFirewallRule -DisplayGroup "Windows Remote Management"

Write-Host ""
Write-Host "Configuring TrustedHosts..."

Set-Item WSMan:\localhost\Client\TrustedHosts -Value $PeerIP -Force

Write-Host ""
Write-Host "Checking WinRM service..."

Get-Service WinRM
```

```
Write-Host ""
Write-Host "Testing WinRM connectivity..."

Test-WSMan $PeerIP

Write-Host ""
Write-Host "=====
Write-Host " WinRM configuration completed successfully"
Write-Host "=====
```

3.7 Server Synchronization Using Robocopy

After completing the initial Windows configuration and establishing PowerShell Remoting, the next stage was to implement the actual server synchronization between Windows01 and Windows02. The purpose of this stage was to make Windows02 a secondary copy of the important data stored on Windows01. If important files on Windows01 were lost or the primary server became unavailable, the secondary server would already contain synchronized copies of the required server data. Server synchronization is required because two Windows VMs being connected to the same network does not mean their files are automatically synchronized. For example, a file created on Windows01:

```
C:\ServerAutomation\example.txt
```

Does not automatically appear on Windows02.

A synchronization mechanism is therefore required to copy the necessary data from the primary server to the secondary server. The project uses Robocopy, a Windows command-line file-copying utility designed for reliable copying and synchronization of directories, for this purpose.

3.8 Identifying the Data to Synchronize

An important part of the implementation was deciding what actually needed to be synchronized. The objective was not to copy the entire Windows operating system. Instead, the synchronization focused on server data and automation files that were relevant to the project. The main directories selected were:

```
C:\inetpub\wwwroot
C:\ServerAutomation
C:\Users\Public
```

C:\inetpub\wwwroot is the physical directory used by the IIS Default Web Site. Synchronizing this directory ensures that the website files on Windows02 can be kept consistent with Windows01.

C:\ServerAutomation contains the PowerShell automation scripts used to manage the Windows environment. It was included because the secondary server should also have access to the automation scripts.

C:\Users\Public contains files intended to be accessible to users on the Windows system. It was included as part of the selected server filesystem data for synchronization.

3.9 The sync_server.ps1 script

The synchronization process was automated using:

```
C:\ServerAutomation\sync_server.ps1
```

The script was designed to:

- Ask for the destination server's IP address
- Define the source and destination directories.
- Check that the source directory exists.
- Check that the destination can be reached.
- Use Robocopy to synchronize the files.
- Check the Robocopy exit code.
- Report whether each synchronization operation succeeded.
- Continue through all selected directories.

This allowed one script to perform the synchronization rather than requiring separate Robocopy commands to be entered manually.

3.10 Complete Script

```
Write-Host "====="
Write-Host " Windows Server Synchronization"
Write-Host "====="

param(
    [Parameter(Mandatory=$true)]
    [string]$DestIP
)

Write-Host ""
Write-Host "Destination Server: $DestIP"

# =====
# IIS Website Synchronization
# =====

$SourcePath = "C:\inetpub\wwwroot"
$DestinationPath = "\\$DestIP\C$\inetpub\wwwroot"

Write-Host ""
Write-Host "-----"
Write-Host "Source:      $SourcePath"
Write-Host "Destination: $DestinationPath"
Write-Host "-----"
Write-Host "Starting synchronization..."
Write-Host ""

robocopy $SourcePath $DestinationPath /E /COPY:DAT /R:3 /W:5

$RobocopyExitCode = $LASTEXITCODE

Write-Host ""

if ($RobocopyExitCode -le 7) {
```

```

        Write-Host "Synchronization successful."
        Write-Host "Robocopy Exit Code: $RobocopyExitCode"
    }
    else {
        Write-Host "Synchronization failed."
        Write-Host "Robocopy Exit Code: $RobocopyExitCode"
    }
}

# =====
# Server Automation Synchronization
# =====

$SourcePath = "C:\ServerAutomation"
$DestinationPath = "\\$DestIP\C$\ServerAutomation"

Write-Host ""
Write-Host "-----"
Write-Host "Source:      $SourcePath"
Write-Host "Destination: $DestinationPath"
Write-Host "-----"
Write-Host "Starting synchronization..."
Write-Host ""

robocopy $SourcePath $DestinationPath /E /COPY:DAT /R:3 /W:5

$RobocopyExitCode = $LASTEXITCODE

Write-Host ""

if ($RobocopyExitCode -le 7) {
    Write-Host "Synchronization successful."
    Write-Host "Robocopy Exit Code: $RobocopyExitCode"
}
else {
    Write-Host "Synchronization failed."
    Write-Host "Robocopy Exit Code: $RobocopyExitCode"
}

# =====
# Public User Data Synchronization
# =====

$SourcePath = "C:\Users\Public"
$DestinationPath = "\\$DestIP\C$\Users\Public"

Write-Host ""
Write-Host "-----"
Write-Host "Source:      $SourcePath"
Write-Host "Destination: $DestinationPath"
Write-Host "-----"
Write-Host "Starting synchronization..."
Write-Host ""

robocopy $SourcePath $DestinationPath /E /COPY:DAT /R:3 /W:5

$RobocopyExitCode = $LASTEXITCODE

Write-Host ""

if ($RobocopyExitCode -le 7) {
    Write-Host "Synchronization successful."
    Write-Host "Robocopy Exit Code: $RobocopyExitCode"
}

```

```

else {
    Write-Host "Synchronization failed."
    Write-Host "Robocopy Exit Code: $RobocopyExitCode"
}

Write-Host ""
Write-Host "=====
Write-Host " Synchronization Complete"
Write-Host "=====
Write-Host "All selected server data has been synchronized."

```

3.11 Installing Required Windows Components

After implementing the server synchronization process, the next stage of the Windows automation was to automate the installation and verification of the required Windows components. In the Linux implementation, this role was performed by the package-installation script, which used the Linux package manager to install required software. Windows Server works differently. Instead of relying primarily on a package manager such as `apt`, many server components are provided as Windows Server Features. For this project, PowerShell was used to inspect and install the required Windows features. The automation was implemented through:

`install_packages.ps1`

3.12 Purpose of `install_packages.ps1`

- Check whether required Windows features are already installed.
- Install missing features.
- Avoid reinstalling components that already exist.
- Display the installation status.
- Provide a repeatable method for preparing the Windows server.

3.13 Complete Script

```

Write-Host "=====
Write-Host " Windows Server Package Installation"
Write-Host "=====

$RequiredFeatures = @(
    "Web-Server"
)

foreach ($Feature in $RequiredFeatures) {

    Write-Host ""
    Write-Host "Checking feature: $Feature"

    $InstalledFeature = Get-WindowsFeature -Name $Feature

    if ($InstalledFeature.Installed) {
        Write-Host "$Feature is already installed."
    }
    else {
        Write-Host "$Feature is not installed."
        Write-Host "Installing $Feature..."

        Install-WindowsFeature -Name $Feature -IncludeManagementTools
    }
}

```

```

        Write-Host "$Feature installation completed."
    }

    Write-Host "Verifying $Feature..."

    $Verification = Get-WindowsFeature -Name $Feature

    if ($Verification.Installed) {
        Write-Host "$Feature verification successful."
    }
    else {
        Write-Host "ERROR: $Feature is not installed."
        exit 1
    }
}

Write-Host ""
Write-Host "======"
Write-Host " Package Verification Complete"
Write-Host "======"

```

3.14 Configuration-Driven Architecture

To achieve a "Zero-Touch" deployment model, the Windows automation was upgraded from interactive scripts (which required manual input via Read-Host) to a configuration-driven architecture. This allows an administrator to deploy and configure a new server simply by editing a configuration file and running a single master script. The environment now utilizes a JSON configuration file named `server_config.json`. This file stores all critical variables, including server roles, IP addresses, and synchronization schedules.

server_config.json Structure:

```

{
    "Role": "PRIMARY",
    "PrimaryIP": "192.168.0.80",
    "SecondaryIP": "192.168.0.81",
    "SyncIntervalMinutes": 10080
}

```

3.15 The Master Orchestrator Script

The `Master-Install.ps1` script acts as the central orchestrator. Instead of containing all the logic, it reads the `server_config.json` file and sequentially executes the sub-scripts (`Setup-Windows.ps1`, `configure_winrm.ps1`, `install_packages.ps1`, and `sync_server.ps1`). Crucially, the sub-scripts were updated to accept Parameters (e.g., `-PeerIP` and `-DestIP`) instead of using `Read-Host`. The Master Script automatically passes the IP addresses from the JSON config file to these sub-scripts, allowing the entire installation to run unattended without pausing for user input.

3.16 Automated Synchronization via Task Scheduler

Previously, synchronization had to be triggered manually. The Master Orchestrator now automatically registers a Windows Task Scheduler job named `AutomatedServerSync` upon completion.

Using PowerShell's `New-ScheduledTaskTrigger` and `Register-ScheduledTask` cmdlets, the script configures the task to run `sync_server.ps1` automatically at the interval defined in the

configuration file (e.g., every 1 week). This ensures that Windows02 remains consistently synchronized with Windows01 without any ongoing manual intervention.

3.17 Complete Script (Master-Install.ps1)

```
# Master-Install.ps1

# 1. Load Configuration
$ConfigPath = Join-Path $PSScriptRoot "server_config.json"
if (-Not (Test-Path $ConfigPath)) {
    Write-Host "ERROR: server_config.json not found in $PSScriptRoot" -
ForegroundColor Red
    exit 1
}
$Config = Get-Content $ConfigPath | ConvertFrom-Json

Write-Host "=====" -ForegroundColor Cyan
Write-Host " Unattended Windows Master Installation"
Write-Host " Role: $($Config.Role) | Peer IP: $($Config.SecondaryIP)"
Write-Host "=====" -ForegroundColor Cyan

# 2. Run Initial Setup (IIS, SSH, Services)
Write-Host "`n[1/4] Running Initial Windows Setup..." -ForegroundColor Yellow
& "$PSScriptRoot\Setup-Windows.ps1"

# 3. Run WinRM Configuration (Passes Peer IP automatically)
Write-Host "`n[2/4] Configuring WinRM and Remoting..." -ForegroundColor Yellow
& "$PSScriptRoot\configure_winrm.ps1" -PeerIP $Config.SecondaryIP

# 4. Run Package Installation (Installs Windows Features)
Write-Host "`n[3/4] Installing Required Windows Components..." -ForegroundColor Yellow
& "$PSScriptRoot\install_packages.ps1"

# 5. Run Initial Sync & Setup Automated Schedule
Write-Host "`n[4/4] Running Initial Sync and Registering Schedule..." -
ForegroundColor Yellow

# Run the sync script once immediately
& "$PSScriptRoot\sync_server.ps1" -DestIP $Config.SecondaryIP

# Register the Windows Task Scheduler to run the sync automatically
$action = New-ScheduledTaskAction -Execute "PowerShell.exe" -Argument "-
ExecutionPolicy Bypass -File `"$PSScriptRoot\sync_server.ps1`" -DestIP
$($Config.SecondaryIP)"
$trigger = New-ScheduledTaskTrigger -Once -At (Get-Date) -RepetitionInterval (New-
TimeSpan -Minutes $Config.SyncIntervalMinutes)

# Register the task (Force replaces it if it already exists)
Register-ScheduledTask -TaskName "AutomatedServerSync" -Action $action -Trigger
$trigger -RunLevel Highest -Force

Write-Host "=====" -ForegroundColor Green
Write-Host " MASTER INSTALLATION COMPLETE!" -ForegroundColor Green
Write-Host " Sync scheduled every $($Config.SyncIntervalMinutes) minutes." -
ForegroundColor Green
Write-Host "=====" -ForegroundColor Green
```


